4. 타입 분석 모듈 상세 분석 문서

개요

`type_analysis` 모듈은 Circom 컴파일러에서 정적 타입 검증과 의미 분석을 담당하는 핵심 모듈입니다. 이 모듈은 파싱된 AST를 받아 신호(Signal), 컴포넌트(Component), 변수(Variable), 버스(Bus)의 타입 정확성을 검증하고, 템플릿 인스턴스화 과정에서의 타입 추론을 수행합니다.

모듈 구조

```
type_analysis/
├── src/
| ├── analyzers/ # 타입 분석 알고리즘
| ├── decorators/ # 타입 추론 및 감소
| ├── check_types.rs # 메인 타입 체킹 인터페이스
| └── lib.rs # 모듈 진입점
```

4.1 신호 타입 시스템 분석

신호 타입 분류

Circom에서 신호는 세 가지 주요 타입으로 분류됩니다:

1. Input 신호

- **역할**: 회로의 입력값을 나타내는 신호

- **특징**:
- 외부에서 값이 제공되어야 함
- `public` 또는 `private` 태그를 가질 수 있음
- 템플릿 인스턴스화 시 값이 할당되어야 함

2. Output 신호

- **역할**: 회로의 출력값을 나타내는 신호
- **특징**:
- 회로 내부에서 계산되어 값이 할당됨
- 다른 컴포넌트의 입력으로 연결될 수 있음
- 제약 조건을 통해 값이 결정됨

3. Intermediate 신호

- **역할**: 회로 내부의 중간 계산 결과를 저장하는 신호
- **특징**:
- 복잡한 계산을 단계별로 분해할 때 사용
- 제약 조건 생성 시 보조 변수 역할
- 외부에서 직접 접근할 수 없음

신호 선언 및 타입 체킹 과정

`check_signal_correctness` 함수 분석

```
pub fn check_signal_correctness(template_data: &TemplateData) -> Result<(), Re
let template_id = template_data.get_file_id();
let template_body = template_data.get_body_as_vec();
```

```
let mut reports = ReportCollection::new();
for stmt in template_body.iter() {
treat_statement(stmt, true, template_id, &mut reports);
}
if reports.is_empty() {
Result::Ok(())
} else {
Result::Err(reports)
}
}
```

주요 검증 과정:

1. **스코프 검증**: 신호는 초기 스코프 또는 알려진 조건의 If 스코프에서만 선언 가능
2. **선언 위치 검증**: While 루프 내부에서는 신호 선언 불가
3. **타입 일관성 검증**: 신호 타입과 할당되는 값의 타입 일치 확인

`treat_statement` 함수의 신호 검증 로직

```
fn treat_statement(
stmt: &Statement,
signal_declaration_allowed: bool,
template_id: FileID,
reports: &mut ReportCollection,
) {
```

```
use Statement::*;
match  stmt {
InitializationBlock { meta, xtype, .. } => match  xtype {
VariableType::Signal(_, _)
| VariableType::Bus(_, _, _)
| VariableType::Component => {
if !signal_declaration_allowed {
let  mut  report = Report::error(
"Signal, bus or component declaration inside While scope. Signals, buses and com
ReportCode::SignalOutsideOriginalScope,
);
// 에러 위치 정보 추가
reports.push(report);
}
}
_ => {}
},
While { stmt, .. } => {
// While 루프 내부에서는 신호 선언 금지
treat_statement(stmt, false, template_id, reports);
}
// 기타 문장 처리...
}
```

```
}
```

`analyse_project` 함수의 타입 검증 알고리즘

메인 타입 분석 함수인 `check_types` 는 다음과 같은 단계적 검증을 수행합니다:

1. 구조적 분석 (Structural Analyses)

```
pub fn check_types(
program_archive: &mut ProgramArchive,
) -> Result<ReportCollection, ReportCollection> {
let mut errors = ReportCollection::new();
let mut warnings = ReportCollection::new();

// 1. 프로그램 레벨 분석
program_level_analyses(program_archive, &mut errors);
if !errors.is_empty() {
return Result::Err(errors);
}

// 2. 템플릿 레벨 분석
template_level_analyses(program_archive, &mut errors);
if !errors.is_empty() {
return Result::Err(errors);
```

```
}

// 3. 함수 레벨 분석
function_level_analyses(program_archive, &mut errors);
// ...
}
```

프로그램 레벨 분석:

- `check_naming_correctness` : 심볼 이름의 정확성 검증
- 전역 스코프에서의 이름 충돌 검사

템플릿 레벨 분석:

- `free_of_returns` : 템플릿 내부에 return 문이 없는지 검증
- `check_signal_correctness` : 신호 선언의 정확성 검증
- `extern_c` 데코레이터 사용 규칙 검증

2. 타입 추론 및 감소 (Type Inference & Reduction)

```
// 템플릿 레벨 데코레이터
template_level_decorators(program_archive, &mut errors);

fn template_level_decorators(
program_archive: &mut ProgramArchive,
reports: &mut ReportCollection,
) {
```

```
// 컴포넌트 타입 추론
component_type_inference::inference(program_archive);
let program_archive2 = program_archive.clone();
for template_data in program_archive.get_mut_templates().values_mut() {
// 타입 감소 수행
reports.append(&mut type_reduction::reduce_template(template_data, &program_arch
}
}
```

3. 핵심 타입 체킹 (type_check)

```
pub fn type_check(program_archive: &ProgramArchive) -> Result<OutInfo, ReportC
let mut analysis_information = AnalysisInformation {
reached: HashSet::new(),
file_id: *program_archive.get_file_id_main(),
reports: ReportCollection::new(),
registered_calls: CallRegister::new(),
environment: TypingEnvironment::new(),
return_type: Option::None,
};
let initial_expression = program_archive.get_main_expression();
let type_analysis_response =
type_expression(initial_expression, program_archive, &mut analysis_information)
// 메인 컴포넌트 태그 검증
```

```
check_main_has_tags(initial_expression, program_archive, &mut  analysis_informat

if  analysis_information.reports.is_empty() {

Result::Ok(OutInfo { reached: analysis_information.reached })

} else {

Result::Err(analysis_information.reports)

}

}
```

신호 차원 및 배열 처리 방법

다차원 신호 선언

```
// AST에서의 신호 선언 처리

Declaration { xtype, name, dimensions, meta, .. } => {

let  typing_response =

type_array_of_expressions(dimensions, program_archive, analysis_information);

let  dimensions_type = typing_response.unwrap_or_default();

// 각 차원 표현식의 타입 검증

for (dim_expression, dim_type) in  dimensions.iter().zip(dimensions_type) {

if  dim_type.is_template() {

add_report(

ReportCode::InvalidArraySizeT,

dim_expression.get_meta(),
```

```
&mut analysis_information.reports,
);
}
else if dim_type.is_bus() {
add_report(
ReportCode::InvalidArraySizeB,
dim_expression.get_meta(),
&mut analysis_information.reports,
);
}
else if dim_type.dim() > 0 {
add_report(
ReportCode::InvalidArraySize(dim_type.dim()),
dim_expression.get_meta(),
&mut analysis_information.reports,
);
}
}
// 신호 타입에 따른 환경 등록
match xtype {
VariableType::Signal(s_type, tags) => {
match s_type {
SignalType::Input => analysis_information
```

```
.environment
.add_input(name, (dimensions.len(), tags.clone())),
SignalType::Output => analysis_information
.environment
.add_output(name, (dimensions.len(), tags.clone())),
SignalType::Intermediate => analysis_information
.environment
.add_intermediate(name, (dimensions.len(), tags.clone())),
}
}
// 기타 타입 처리...
}
}
```

배열 접근 타입 검증

```
// 배열 인라인 선언 타입 체킹
ArrayInLine { meta, values } => {
let values_types =
type_array_of_expressions(values, program_archive, analysis_information)?;
if values_types.is_empty() {
return add_report_and_end(
ReportCode::EmptyArrayInlineDeclaration,
meta,
```

```
&mut analysis_information.reports,
);
}
let inferred_dim = values_types[0].dim();
for (expression, value_type) in values.iter().zip(values_types.iter()) {
if value_type.is_template() {
add_report(
ReportCode::InvalidArrayType,
expression.get_meta(),
&mut analysis_information.reports,
);
} else if inferred_dim != value_type.dim() {
add_report(
ReportCode::NonHomogeneousArray(inferred_dim, value_type.dim()),
expression.get_meta(),
&mut analysis_information.reports,
);
}
}
Result::Ok(FoldedType::arithmetic_type(inferred_dim + 1))
}
```

타입 환경 (Typing Environment) 관리

환경 구조

```
type ComponentInfo = (Option<String>, ArithmeticType);

type BusInfo = (Option<String>, ArithmeticType, std::vec::Vec<std::string::Strin

type SignalInfo = (ArithmeticType, std::vec::Vec<std::string::String>);

type VarInfo = ArithmeticType;

type TypingEnvironment = CircomEnvironment<ComponentInfo, SignalInfo, VarInfo, B
```

환경 정보 구성:

- **ComponentInfo**: (템플릿 이름, 차원 수)
- **SignalInfo**: (차원 수, 태그 목록)
- **VarInfo**: 차원 수
- **BusInfo**: (버스 타입 이름, 차원 수, 태그 목록)

신호 할당 타입 검증

```
Substitution { var, access, op, rhe, meta, .. } => {

let  rhe_response = type_expression(rhe, program_archive, analysis_information);

let  rhe_type = if  let Result::Ok(r_type) = rhe_response {

r_type

} else {

return;

};
```

```
// 할당 연산자와 심볼 타입 일치성 검증
match (&symbol_information, op) {
(SymbolInformation::Signal(_), AssignOp::AssignConstraintSignal)
| (SymbolInformation::Signal(_), AssignOp::AssignSignal)
| (SymbolInformation::Var(_), AssignOp::AssignVar)
| (SymbolInformation::Component(_), AssignOp::AssignVar) => {}
(SymbolInformation::Signal(_), AssignOp::AssignVar) => {
return add_report(
ReportCode::WrongTypesInAssignOperationOperatorSignal,
meta,
&mut analysis_information.reports,
);
}
_ => {
return add_report(
ReportCode::WrongTypesInAssignOperationOperatorNoSignal,
meta,
&mut analysis_information.reports,
);
}
}
// 차원 일치성 검증
match symbol_information {
```

```
SymbolInformation::Signal(dim) | SymbolInformation::Var(dim) => {
if  dim != rhe_type.dim() {
add_report(
ReportCode::WrongTypesInAssignOperationDims(dim, rhe_type.dim()),
meta,
&mut  analysis_information.reports,
)
}
}
// 기타 타입 처리...
}
}
```

에러 리포팅 시스템

타입 분석 과정에서 발생하는 주요 에러 타입들:

신호 관련 에러

- **`SignalOutsideOriginalScope`** : While 루프 내부에서의 신호 선언
- **`InvalidArraySize`** : 잘못된 배열 크기 지정
- **`WrongTypesInAssignOperationOperatorSignal`** : 신호에 잘못된 할당 연산자 사용
- **`WrongTypesInAssignOperationDims`** : 차원 불일치

배열 관련 에러

- **`EmptyArrayInlineDeclaration`** : 빈 배열 인라인 선언

- **NonHomogeneousArray** : 비동질적 배열 (서로 다른 타입의 원소)
- **InvalidArrayType** : 배열에 템플릿 타입 사용

실제 사용 예시

올바른 신호 선언

```
template Example() {
signal input a; // Input 신호
signal output b; // Output 신호
signal intermediate c; // Intermediate 신호
signal input arr[10]; // 1차원 배열 신호
signal output matrix[3][4]; // 2차원 배열 신호
c <== a * 2; // 제약 조건 할당
b <== c + 1; // 출력 신호 할당
}
```

타입 에러 예시

```
template ErrorExample() {
signal input a;
for (var i = 0; i < 10; i++) {
signal temp; // 에러: While/For 루프 내부에서 신호 선언 불가
}
a = 5; // 에러: 신호에 일반 할당 연산자 사용 불가
```

```
}
```

이러한 타입 분석 시스템을 통해 Circom 컴파일러는 컴파일 타임에 신호 사용의 정확성을 보장하고, 런타임 에러를 방지합니다.

4.

2 템플릿 매개변수 타입 추론 분석

템플릿 매개변수 시스템 개요

Circom에서 템플릿은 매개변수화된 회로 정의를 제공합니다. 템플릿 매개변수는 컴파일 타임에 구체적인 값으로 인스턴스화되어야 하며, 타입 분석 모듈은 이 과정에서 매개변수의 타입 정확성을 검증합니다.

컴포넌트 타입 추론 (`component_type_inference`)

추론 과정 개요

```
pub fn inference(program_archive: &mut ProgramArchive) {
let mut template_to_inference = HashMap::new();
for (name, data) in &program_archive.templates {
let mut analysis =
PathInformation { components: HashSet::new(), environment: HashMap::new() };
infer_component_types(data.get_body(), &program_archive.templates, &mut analysi
template_to_inference.insert(name.clone(), analysis.environment);
}

for (name, mut inference) in template_to_inference {
```

```
let body = program_archive.get_mut_template_data(&name).get_mut_body();
apply_inference(body, &mut inference);
}
}
```

추론 단계:

1. **분석 단계**: 각 템플릿의 본문을 순회하며 컴포넌트 할당 패턴 분석
2. **적용 단계**: 추론된 타입 정보를 AST 메타데이터에 적용

컴포넌트 타입 추론 알고리즘

```
fn infer_component_types(stmt: &Statement, templates: &TemplateInfo, data: &mut
use Statement::*;
match stmt {
// 컴포넌트 선언 감지
Declaration { xtype, name, .. }
if VariableType::Component == *xtype || VariableType::AnonymousComponent == *xty
data.components.insert(name.clone());
}
// 컴포넌트 할당에서 템플릿 타입 추론
Substitution { var, rhe, .. } if data.components.contains(var) => {
if let Some(template) = into_template_inference(rhe, templates) {
data.environment.insert(var.clone(), template);
}
```

```
}
// 제어 구조 재귀 처리
IfThenElse { if_case, else_case, .. } => {
infer_component_types(if_case, templates, data);
if let Some(else_stmt) = else_case {
infer_component_types(else_stmt, templates, data);
}
}
While { stmt, .. } => {
infer_component_types(stmt, templates, data);
}
Block { stmts, .. } => {
for s in stmts {
infer_component_types(s, templates, data);
}
}
InitializationBlock { initializations, .. } => {
for s in initializations {
infer_component_types(s, templates, data);
}
}
_ => {}
}
```

```
}
```

템플릿 호출 패턴 인식

```
fn into_template_inference(expr: &Expression, templates: &TemplateInfo) -> Opti
use Expression::*;
match expr {
// 직접 템플릿 호출
Call { id, .. } if templates.contains_key(id) => Some(id.clone()),
// 조건부 템플릿 선택
InlineSwitchOp { if_true, if_false, .. } => {
let mut ret = into_template_inference(if_true, templates);
if ret.is_none() {
ret = into_template_inference(if_false, templates);
}
ret
}
// 병렬 연산자
ParallelOp { rhe, .. } => {
into_template_inference(rhe, templates)
},
_ => None,
}
```

```
}
```

지원되는 패턴:

- **직접 호출**: `component c = Template(args);`
- **조건부 선택**: `component c = condition ? Template1(args) : Template2(args);`
- **병렬 연산**: `component c = parallel Template(args);`

추론 결과 적용

```
fn apply_inference(stmt: &mut Statement, env: &mut Environment) {
use Statement::*;
match stmt {
Declaration { xtype, name, meta, .. }
if VariableType::Component == *xtype || VariableType::AnonymousComponent == *xty
// 추론된 템플릿 타입을 메타데이터에 저장
meta.component_inference = env.remove(name);
}
// 재귀적으로 모든 문장 처리
IfThenElse { if_case, else_case, .. } => {
apply_inference(if_case, env);
if let Some(else_stmt) = else_case {
apply_inference(else_stmt, env);
}
}
```

```
// 기타 문장 타입들...

_ => {}

}

}
```

타입 감소 (Type Reduction) 시스템

타입 감소 개념

타입 감소는 복잡한 표현식을 단순한 타입 정보로 변환하는 과정입니다. 이를 통해 컴파일러는 각 표현식이 어떤 종류의 값(신호, 컴포넌트, 버스 등)을 나타내는지 파악할 수 있습니다.

```
pub fn reduce_template(template_data: &mut TemplateData, program_archive: &Pro

let mut environment = Environment::new();

// 템플릿 매개변수를 환경에 추가

for param in template_data.get_name_of_params() {

environment.add_variable(param, ());

}

let body = template_data.get_mut_body();

reduce_types_in_statement(body, &mut environment, program_archive)

}
```

타입 감소 분류

```
#[derive(Clone)]

pub enum TypeReduction {
```

```
Variable, // 일반 변수
Signal, // 신호
Component(Option<String>), // 컴포넌트 (템플릿 이름)
Bus(Option<String>), // 버스 (버스 타입 이름)
Tag, // 태그 (신호/버스의 속성)
}
```

표현식 타입 감소

```
fn  reduce_types_in_expression(expression: &mut Expression, environment: &Enviro
-> ReportCollection {
use Expression::*;
match  expression {
// 변수 접근 타입 감소
Variable { name, access, meta, .. } => {
reduce_types_in_variable(name, environment, access, meta, program_archive)
}
// 템플릿 호출 타입 감소
Call { args, id, meta, .. } => {
meta.get_mut_type_knowledge().set_reduces_to(TypeReduction::Component(Some(id.cl
reduce_types_in_vec_of_expressions(args, environment, program_archive)
},
// 버스 호출 타입 감소
```

```
BusCall { args, meta, id, .. } => {
meta.get_mut_type_knowledge().set_reduces_to(TypeReduction::Bus(Some(id.clone()))
reduce_types_in_vec_of_expressions(args, environment, program_archive)
},
// 병렬 연산자 타입 전파
ParallelOp { meta, rhe, .. } => {
let  report = reduce_types_in_expression(rhe, environment, program_archive);
meta.get_mut_type_knowledge().set_reduces_to(rhe.get_meta().get_type_knowledge()
report
}
// 산술 표현식들은 기본적으로 Variable 타입
Number(..) => Vec::new(),
InfixOp { lhe, rhe, .. } => reduce_types_in_infix(lhe, rhe, environment, program
// 기타 표현식들...
}
}
```

변수 접근 타입 감소

```
fn  reduce_types_in_variable(
oname: &str,
environment: &Environment,
access: &mut [Access],
```

```
meta: &mut Meta,
program_archive: &ProgramArchive
) -> ReportCollection {
use Access::*;
use TypeReduction::*;
let mut reports = Vec::new();
// 초기 타입 결정
let mut reduction = if environment.has_signal(oname) {
Signal
} else if environment.has_component(oname) {
environment.get_component(oname).unwrap().get_reduces_to()
} else if environment.has_bus(oname) {
environment.get_bus(oname).unwrap().get_reduces_to()
} else {
Variable
};


// 접근 패턴에 따른 타입 변환
for acc in access {
match acc {
ComponentAccess(name) => {
match reduction {
Component(ref comp) => {
```

```
if  let Some(comp) = comp {
let  template = program_archive.get_template_data(comp.as_str());
let  wire = template.get_inputs().get(name).or(template.get_outputs().get(name))
if  wire.is_some() {
match  wire.unwrap().get_type() {
WireType::Signal => reduction = Signal,
WireType::Bus(new_name) => reduction = Bus(Some(new_name)),
}
} else {
name_not_found_in_component_error(name.clone(), oname.to_string(), meta, &mut  r
return  reports;
}
}
},
Bus(ref  b) => {
if  let Some(b) = b {
let  busdata = program_archive.get_bus_data(b.as_str());
if  let Some(wire) = busdata.get_fields().get(name) {
match  wire.get_type() {
WireType::Signal => reduction = Signal,
WireType::Bus(new_name) => reduction = Bus(Some(new_name)),
}
} else {
```

```
reduction = Tag;
}
}
},
Signal => reduction = Tag,
Tag => {},
Variable => {}, // 타입 체킹에서 에러 처리
}
},
ArrayAccess(exp) => {
reports.append(&mut  reduce_types_in_expression(exp, environment, program_archiv
},
}
}
meta.get_mut_type_knowledge().set_reduces_to(reduction);
reports
}
```

템플릿 인스턴스화 시 타입 검증

컴포넌트 할당 검증

```
// type_check.rs에서의 컴포넌트 할당 검증
Substitution { var, access, op, rhe, meta, .. } => {
```

```
let rhe_response = type_expression(rhe, program_archive, analysis_information);
let rhe_type = if let Result::Ok(r_type) = rhe_response {
r_type
} else {
return;
};


match symbol_information {
SymbolInformation::Component(possible_template) => {
if rhe_type.is_template() {
if possible_template.is_none() {
// 첫 번째 할당: 템플릿 타입 설정
let (current_template, _) = analysis_information
.environment
.get_mut_component_or_break(var, file!(), line!());
*current_template = rhe_type.template;
} else {
// 후속 할당: 타입 일치성 검증
let template = possible_template.unwrap();
let r_template = rhe_type.template.unwrap();
if template != r_template {
add_report(
ReportCode::WrongTypesInAssignOperationArrayTemplates,
```

```
meta,
&mut analysis_information.reports,
)
}
}
} else {
add_report(
ReportCode::WrongTypesInAssignOperationTemplate,
meta,
&mut analysis_information.reports,
)
}
}
// 기타 심볼 타입들...
}
}
```

타입 레지스터 시스템

함수 타입 추론

```
pub struct TypeInstance<Type> {
pub argument_dimensions: Vec<Type>,
pub returned_dimension: Type,
```

```
}

pub struct TypeRegister<Type> {
pub id_to_instances: HashMap<String, Vec<TypeInstance<Type>>>,
}

impl<Type: Default + Eq> TypeRegister<Type> {
pub fn get_instance(&self, id: &str, look_for: &[Type]) -> Option<&TypeInstanc
if !self.id_to_instances.contains_key(id) {
return Option::None;
}
let instances = self.id_to_instances.get(id).unwrap();
for instance in instances {
if instance.arguments() == look_for {
return Option::Some(&instance);
}
}
Option::None
}
pub fn add_instance(
&mut self,
id: &str,
argument_dimensions: Vec<Type>,
```

```
returned_dimension: Type,
) {
if  self.get_instance(id, &argument_dimensions).is_some() {
return;
}
// 새로운 타입 인스턴스 추가
if !self.id_to_instances.contains_key(id) {
self.id_to_instances.insert(id.to_string(), Vec::new());
}
if  let Option::Some(instances) = self.id_to_instances.get_mut(id) {
let  instance = TypeInstance { argument_dimensions, returned_dimension };
instances.push(instance);
}
}
}
```

함수 반환 타입 추론

```
pub  fn  type_given_function(
function_name: &str,
function_info: &HashMap<String, FunctionData>,
params_types: &[Type],
) -> Option<Type> {
let  mut  explored_functions = NodeRegister::new();
```

```
start(function_name, &mut  explored_functions, function_info, params_types)

}



fn  start(

function_name: &str,

explored_functions: &mut NodeRegister,

function_info: &HashMap<String, FunctionData>,

params_types: &[Type],

) -> Option<Type> {

let  function_data = function_info.get(function_name).unwrap();

let  mut  environment = Environment::new();

let  mut  initial_block = Block::new();

// 순환 호출 방지

explored_functions.insert(function_name.to_string());

// 매개변수 타입 설정

for (name, t) in  function_data.get_name_of_params().iter().zip(params_types.ite

initial_block.insert(name.clone(), t.clone());

}

environment.push(initial_block);

look_for_return_value(

function_name,

&mut  environment,

explored_functions,
```

```
function_data,
function_info,
)
}
```

타입 오류 탐지 및 리포팅 메커니즘

주요 타입 오류 분류

1. **템플릿 관련 오류**

- `WrongTypesInAssignOperationTemplate` : 컴포넌트에 비템플릿 값 할당
- `WrongTypesInAssignOperationArrayTemplates` : 배열 컴포넌트 타입 불일치
- `InvalidSignalAccess` : 컴포넌트에서 존재하지 않는 신호 접근

2. **차원 관련 오류**

- `WrongTypesInAssignOperationDims` : 배열 차원 불일치
- `InvalidArraySize` : 잘못된 배열 크기 지정
- `NonHomogeneousArray` : 비동질적 배열

3. **타입 불일치 오류**

- `MustBeArithmetic` : 산술 표현식이 필요한 곳에 템플릿 사용
- `MustBeSameDimension` : 제약 조건에서 차원 불일치
- `MustBeBus` : 버스가 필요한 곳에 다른 타입 사용

에러 리포팅 예시

```
fn name_not_found_in_component_error(signal: String, what: String, meta: &Meta,
```

```
let message = "Bus or signal not defined in component".to_string();

let error_code = ReportCode::InvalidSignalAccess;

let mut report = Report::error(message, error_code);

let message = signal + &" is not defined in ".to_string() + what.as_str();

report.add_primary(meta.file_location(), meta.get_file_id(), message);

reports.push(report);

}
```

제네릭 템플릿 처리 방법

매개변수화된 템플릿 예시

```
template GenericMultiplier(n) {

signal input a[n];

signal input b[n];

signal output c[n];

for (var i = 0; i < n; i++) {

c[i] <== a[i] * b[i];

}

}


component mult4 = GenericMultiplier(4);

component mult8 = GenericMultiplier(8);
```

템플릿 매개변수 검증

```
// 템플릿 호출 시 매개변수 타입 검증
Call { args, id, meta, .. } => {
if program_archive.contains_template(id) {
let template_data = program_archive.get_template_data(id);
let expected_params = template_data.get_num_of_params();
if args.len() != expected_params {
add_report(
ReportCode::WrongNumberOfParameters(expected_params, args.len()),
meta,
&mut analysis_information.reports,
);
}
// 각 매개변수의 타입 검증
for arg in args {
let arg_type = type_expression(arg, program_archive, analysis_information)?;
if arg_type.is_template() || arg_type.is_bus() {
add_report(
ReportCode::InvalidTemplateParameter,
arg.get_meta(),
&mut analysis_information.reports,
);
```

```
}
}
Result::Ok(FoldedType::template(id))
}
}
```

실제 사용 예시

올바른 템플릿 매개변수 사용

```
template Multiplier(n) {
signal input a[n];
signal input b[n];
signal output c[n];
for (var i = 0; i < n; i++) {
c[i] <== a[i] * b[i];
}
}

template Main() {
component mult = Multiplier(4); // 올바른 매개변수 전달
// 타입 추론이 정확히 수행됨
mult.a[0] <== 10;
mult.b[0] <== 20;
```

```
// mult.c[0]는 자동으로 200이 됨

}
```

타입 오류 예시

```
template ErrorExample() {

component mult1 = Multiplier(); // 에러: 매개변수 누락

component mult2 = Multiplier(mult1); // 에러: 컴포넌트를 매개변수로 사용

component mult3 = Multiplier([1,2,3]); // 에러: 배열을 매개변수로 사용

mult1.nonexistent <== 5; // 에러: 존재하지 않는 신호 접근

}
```

이러한 포괄적인 타입 추론 및 검증 시스템을 통해 Circom 컴파일러는 템플릿 인스턴스화 과정에서 발생할 수 있는 다양한 타입 오류를 컴파일 타임에 탐지하고 개발자에게 명확한 오류 메시지를 제공합니다.